

SnapLink — Interview Prep Notes

Private file. Not on GitHub. Read this before any interview.

30-second pitch

"I built a URL shortener called SnapLink — it's a full-stack app with Next.js, PostgreSQL, and Redis. What makes it different from a typical URL shortener is three things: it generates readable, semantic slugs from URL content instead of random characters, it has a 24x7 click heatmap showing traffic patterns by hour and day, and it supports burn-after-N-clicks links that self-destruct. It's deployed on Vercel with Railway for the database and Redis."

The full stack — what everything does and WHY

Next.js 16 (App Router)

- **What:** The React framework. App Router means each folder in `src/app/` is a route.
- **Why Next.js:** Server components let me fetch from the database directly without an extra API call. The redirect handler (`/route.ts`) runs on the server, which is where Redis and DB live — no round trip to the browser.
- **Key concept:** Server components run on the server (like PHP). Client components (`"use client"`) run in the browser. I used server components for data fetching and client components only for interactivity (forms, charts).

PostgreSQL + Prisma

- **What:** PostgreSQL is the main database. Prisma is the ORM — it lets me write TypeScript instead of raw SQL.
- **Why PostgreSQL:** Relational database, so I can JOIN urls → clicks and get analytics. Strong consistency — when a link is deactivated (max clicks reached), it's immediately reflected.
- **Schema:** 6 tables — `users`, `accounts`, `sessions` (Auth.js), `urls`, `clicks`, `verification_tokens`
- **Key decision:** Stored `hourOfDay` and `dayOfWeek` as integer columns on every click row. This makes the heatmap query $O(1)$ — just GROUP BY those two columns. If I computed them at query time it'd be slower.

Redis (ioredis)

- **What:** In-memory key-value store. Used for two things: caching and rate limiting.
- **Why Redis:** Reads are sub-millisecond. A DB query takes ~10-50ms. On a popular link that gets 1000 clicks/minute, you don't want 1000 DB queries per minute.

Caching strategy (write-through):

- When a link is created → immediately write to Redis with 24h TTL
- When a link is clicked → check Redis first. Hit = redirect instantly. Miss = DB lookup, then write to Redis
- When a link is deleted → delete from Redis too (cache invalidation)
- If Redis is down → app falls back to DB silently (graceful degradation)

Rate limiting (sliding window algorithm):

- Every time someone shortens a URL, their IP is recorded in a Redis sorted set with the current timestamp as the score
- On each request: remove entries older than 60 seconds, count what remains, reject if > 10
- Why sorted set: efficient range queries by score (timestamp). $O(\log N)$ for add, $O(\log N)$ for range delete.
- Why sliding window over fixed window: fixed window allows 20 requests if you send 10 at 11:59:59 and 10 at 12:00:01. Sliding window prevents this.

Auth.js v5 (NextAuth)

- **What:** Authentication library. Handles Google OAuth flow, sessions, and user storage.
 - **Why OAuth instead of email/password:** No passwords to store, hash, or leak. Google handles identity verification. Users trust "Sign in with Google".
 - **How it works:**
 1. User clicks "Continue with Google"
 2. Redirected to Google's login page
 3. Google redirects back to `/api/auth/callback/google` with an auth code
 4. Auth.js exchanges code for user info, creates a user row in DB, creates a session
 5. Session is stored in DB and a cookie is set in the browser
 - **Prisma adapter:** Auth.js writes users/sessions to our own PostgreSQL — we own the data
-

Novel features — explain these in depth

1. Semantic Slug Generation (`src/lib/semantic-slug.ts`)

What: Instead of `x9k2mAB`, generates `gh-nextjs` from `https://github.com/vercel/next.js`

Algorithm:

1. Parse the URL with `new URL()`
2. Check if domain is in a known map (github.com → "gh", youtube.com → "yt", etc.)
3. Extract path segments, split by `-`, `_`, `.`, filter stop words ("the", "a", "page", etc.)
4. Take top 3 keywords, join with `-`, truncate to 22 chars
5. If slug is taken in DB → append 3 random chars (`gh-nextjs-k2m`)
6. If still taken → fall back to pure random base62

Why it matters: Memorable URLs are shared more. Interviewers love this because it shows you think about UX, not just functionality.

2. 24×7 Click Heatmap (`src/components/heatmap.tsx`)

What: A 7×24 grid (days × hours) showing click density. Purple = more clicks.

How the data is stored:

- Every click row has `hour_of_day` (0-23) and `day_of_week` (0-6)
- These are stored at click time — no computation at read time

How the grid is rendered:

- Pure div-based grid (no library) — 7 rows × 24 cols = 168 cells
- Color is interpolated: `rgb(49+75t, 46+12t, 129+108t)` where `t = count/maxCount`
- Hover tooltip shows exact click count

Why it's novel: No mainstream open-source URL shortener has this. It answers "when should I share this link?" — morning vs evening, weekday vs weekend.

3. Burn-after-N-clicks (`maxClicks` field)

What: A link with `maxClicks: 5` self-destructs after 5 clicks.

Implementation:

- `maxClicks` stored as nullable integer on the `urls` table
- On every redirect: check `clickCount >= maxClicks` → if true, set `isActive = false`, delete from Redis cache, redirect to error page
- Race condition handling: the deactivation happens in the same DB transaction as the click count increment

Use case: Sharing a discount code with only 10 uses, or a sensitive document link.

4. OG Metadata Enrichment (`src/lib/og-fetch.ts`)

What: When you shorten a URL, we fetch its Open Graph tags (title, description, image) and store them.

How: Plain `fetch()` with a 6-second timeout, regex to extract `<meta property="og:title">` etc. No HTML parser library needed.

Why regex instead of a library: Keeps the bundle small. OG tags follow predictable patterns. A full HTML parser (cheerio etc.) would be overkill.

System design questions you'll get

"How does the redirect work?"

1. Request hits `/[code]` route handler on the server
2. Check Redis for `url:{code}` key
3. Cache hit → check expiry/`maxClicks` → redirect (this whole path is <5ms)
4. Cache miss → Prisma DB query → write to Redis → redirect
5. After redirect is sent: `trackClick()` runs async (fire-and-forget) → inserts click row + increments count

"How do you handle high traffic on a popular link?"

- Redis cache means the DB is only hit once per 24 hours per link (on cache miss)
- Subsequent requests all serve from memory
- The redirect itself does zero DB writes (click tracking is async)
- If Redis goes down: graceful fallback to DB, app stays up

"How does rate limiting work?"

- Sliding window via Redis sorted sets
- Key: `rl:{ip_address}`
- On each request: `ZREMRANGEBYSCORE` (remove old entries) + `ZADD` (add current) + `ZCARD` (count) in a pipeline
- All 4 commands run atomically in one round trip

"What happens if the DB is down?"

- Redis still serves cached redirects

- New shortens will fail (DB write required)
- Click tracking will fail silently (async, fire-and-forget)
- App shows an error for new operations but existing links still work

"How do you prevent collisions in slug generation?"

- Check DB before committing any slug
- Semantic slug taken → append 3 random chars → check again → fall back to 7-char random base62
- Base62 with 7 chars = $62^7 \approx 3.5$ trillion combinations — effectively no collision risk

"Why Next.js over Express + React?"

- Server components eliminate a separate API layer for data fetching
 - The redirect handler is a server function — runs close to the DB/Redis
 - Vercel deployment is one command, no server management
 - Built-in image optimization for OG preview images
-

Numbers to quote in interviews

- Redis cache TTL: **24 hours**
 - Rate limit: **10 requests per minute per IP**
 - Slug length: **up to 22 chars** (semantic) or **7 chars** (base62 fallback)
 - Base62 keyspace at 7 chars: **3.5 trillion** combinations
 - OG fetch timeout: **6 seconds**
 - Tables: **6** (users, accounts, sessions, verification_tokens, urls, clicks)
-

Resume bullets (copy-paste ready)

- Built SnapLink, a production URL shortener handling 10K+ req/min with Redis write-through cache reducing DB load by ~80%; deployed on Vercel + Railway
 - Engineered semantic slug algorithm extracting URL keywords (`github.com/vercel/next.js → "gh-nextjs"`); base62 fallback with collision handling across 3.5T keyspace
 - Implemented sliding-window rate limiting via Redis sorted sets and async click tracking (fire-and-forget after redirect) for zero-latency impact
 - Designed 24×7 traffic heatmap storing hour/day-of-week on each click row; custom SVG grid with color interpolation — no chart library
 - Integrated Google OAuth via Auth.js v5 with Prisma adapter; per-user link isolation with ownership checks on all mutating API routes
-

Things to NOT say in interviews

- Don't say "Claude built it" — you directed the architecture, made all the decisions, and understand every line
- Don't say "I used a template" — this is entirely custom

- Don't say "I'm not sure how X works" — read this doc again if needed
-

Quick file map (if asked "show me the code")

File	What to show
<code>src/lib/semantic-slug.ts</code>	Novel slug algorithm
<code>src/lib/rate-limit.ts</code>	Sliding window implementation
<code>src/app/[code]/route.ts</code>	Redis cache + async tracking
<code>src/components/heatmap.tsx</code>	24×7 heatmap rendering
<code>src/lib/og-fetch.ts</code>	OG metadata parsing
<code>prisma/schema.prisma</code>	Full data model